AI Security Findings

Overall risk: LOW | Total findings: 1

Executive summary

Risk overview

Total findings: 1. Critical: 0, High: 1, Medium: 0, Low: 0.

Technical summary

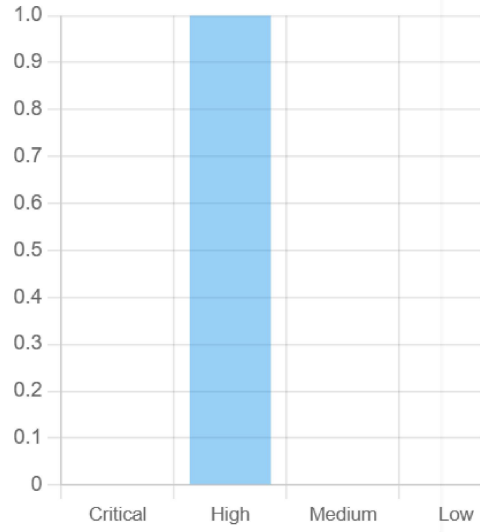
Overall Risk Level: High

Key Issues to Fix First:
1. SQL Injection Vulnerability: The 'get_user' function directly interpolates unsanitized user input into an SQL query, posing a serious risk of SQL injection.
2. Unsafe Query Construction: The approach of embedding user inputs into SQL queries without parameterization makes the system vulnerable not only in this instance but potentially elsewhere if similar practices are adopted.

Quick Wins and Hardening Recommendations:
• Immediately refactor the vulnerable code to use parameterized queries or prepared statements. For example, switch to a safe method such as conn.execute("SELECT * FROM users WHERE name = ?", (username,)) to handle user input securely.
• Conduct a thorough code review to identify other areas where user input may be directly concatenated into SQL statements or commands.
• Implement comprehensive security testing, such as automated static analysis and dynamic penetration tests, to ensure that similar vulnerabilities are not present elsewhere in the application.
• Consider adding input validation and sanitization as an additional layer of security to reduce the risk of injection attacks.

These measures will significantly strengthen your codebase against common injection attacks and improve overall application security.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
HIGH	SQL Injection Vulnerability	`query construction in the get_user function`	User-supplied input from 'username' in the request is directly interpolated into the SQL query string without proper sanitization, allowing an attacker to inject malicious SQL code.	Use parameterized queries or prepared statements to safely handle user input. For example, use 'conn.execute("SELECT * FROM users WHERE name = ?", (username,))' to mitigate the risk of SQL injection.